

Tutorial - Semana 7 - Encontro 1

Introdução

Desde a Semana 6, cada grupo já coleta itens de `DT_Items` através do Actor de coleta (`BP_Chest` / `BP_Pickup`), disparando o Event Dispatcher construído na Semana 5. Mas esse progresso existe apenas enquanto o jogo está rodando: ao fechar o editor ou o build, tudo se perde. Este encontro resolve esse problema com o `SaveGame Object` — a classe da Unreal dedicada a serializar estado entre sessões — e centraliza toda a leitura/escrita em um `SaveComponent` , conforme `PROJECT_ARCHITECTURE.md`. Ao final, o progresso de coleta de itens sobrevive ao fechamento e reabertura do nível.

Este tutorial não substitui a explicação do professor em sala. Ele existe para que você possa acompanhar a implementação passo a passo durante o laboratório e revisitar os passos depois da aula, sem depender da documentação oficial da Epic.

Objetivos da Semana

- Explicar o `SaveGame Object` como mecanismo de serialização e recuperação de estado de jogo.
- Comparar `SaveGame Object` com `PlayerPrefs` e serialização própria em JSON na Unity.
- Implementar `BP_SaveGame` e `SaveComponent` para gravar e recuperar o progresso de coleta de itens.

Resultado Esperado ao Final da Semana

`BP_SaveGame` e `SaveComponent` funcionais, gravando o progresso de coleta de itens de `DT_Items` no momento em que um item é coletado, e recuperando esse progresso ao carregar o nível — validado por um teste de fechar e reabrir o nível.

Pré-requisitos

- `DT_Items` (Semana 6), tipada por `S_ItemData` / `E_ItemType` .
 - `BPI_Interactable` e Event Dispatcher de interação (Semana 5) funcionais.
 - Actor de coleta (`BP_Chest` / `BP_Pickup`) consultando `DT_Items` e disparando o Event Dispatcher ao ser interagido.
-

Antes de começar

O que você deverá possuir antes desta semana

- O projeto da Semana 6, com `DT_Items` tipada e populada, e ao menos um Actor de coleta funcional no nível de teste.

Arquivos necessários

- Nenhum arquivo externo é necessário neste encontro.

Assets utilizados

- Nenhum asset novo obrigatório.

Projeto esperado

O projeto da Semana 6, com o Actor de coleta consultando `DT_Items` e disparando o Event Dispatcher da Semana 5 ao ser interagido.

Parte 1 — SaveGame Object: serializando estado entre sessões

Objetivo

Compreender o SaveGame Object como mecanismo de persistência e criar `BP_SaveGame`, a classe responsável por guardar o progresso de coleta de itens.

Conceito

Até aqui, todo o progresso do jogador existe apenas em memória, enquanto o jogo está rodando. Ao fechar o editor ou o build, esse estado desaparece — o jogador não pode "retomar de onde parou".

Toda engine madura precisa de um mecanismo que capture uma fração do estado em memória e a grave em um formato recuperável em disco.

A Unreal resolve isso com o **SaveGame Object**: uma classe que existe fora do fluxo normal de Actors, pensada exclusivamente para ser serializada e desserializada. Diferente de um Actor, um objeto de SaveGame não existe no nível, não é renderizado e não recebe Tick — sua única responsabilidade é guardar propriedades marcadas para persistência e ser gravado/lido através das funções nativas de Save/Load da engine.

O princípio arquitetural reforçado por PROJECT_ARCHITECTURE.md é que **nenhum Actor deve implementar sua própria lógica de leitura/escrita de arquivo**. Essa responsabilidade é centralizada em um único ponto — o `SaveComponent`, construído na Parte 2 — evitando que a lógica de serialização se espalhe pelo projeto à medida que novos sistemas persistentes forem adicionados (como o Inventário, na Semana 10).

Passo a passo

1. No Content Browser, navegar até a subpasta `Data/` (ou criar uma subpasta específica para SaveGame, conforme a organização do grupo).
2. Clicar com o botão direito e selecionar **Blueprint Class**.
3. Na busca de classe-pai, procurar por `SaveGame` e selecioná-la como base.
4. Nomear o novo Blueprint `BP_SaveGame`, conforme a convenção `BP_`.
5. Abrir `BP_SaveGame` e criar uma variável do tipo Array de `Name` (ou `String`), chamada `ItensColetados`, que guardará os identificadores (Row Name) dos itens já coletados.
6. Compilar e salvar `BP_SaveGame`.

Resultado esperado

Uma classe `BP_SaveGame`, derivada de `SaveGame`, com uma propriedade `ItensColetados` pronta para armazenar os identificadores dos itens coletados pelo jogador.

Verificando

Abrir `BP_SaveGame` e confirmar, no painel Class Settings, que a classe-pai é `SaveGame` (não `Actor` nem nenhuma outra classe), e que a variável `ItensColetados` está visível na aba My Blueprint com o tipo Array correto.

Problemas comuns

- Criar `BP_SaveGame` a partir de `Actor` em vez de `SaveGame` : um `Actor` com essas variáveis não ganha suporte nativo de serialização; a classe-pai precisa ser especificamente `SaveGame` .
- Guardar dados demais dentro de `BP_SaveGame` neste momento: o escopo deste encontro é apenas o progresso de coleta de itens; outros dados (posição do jogador, vida) pertencem a etapas futuras da disciplina.

Boas práticas

Mantenha `BP_SaveGame` como uma estrutura de dados simples — apenas propriedades, sem lógica de gameplay. A lógica de quando e como gravar/ler pertence ao `SaveComponent` , não à classe de `SaveGame` em si.

Comparação com Unity

A Unity resolve o mesmo problema de persistência principalmente de duas formas: `PlayerPrefs` , adequado apenas a dados simples e pequenos (chave-valor), e serialização própria em JSON (via `JsonUtility` ou bibliotecas externas) para estados mais complexos, exigindo que o próprio desenvolvedor defina o formato e a gravação em arquivo. A Unreal já oferece uma classe dedicada (`USaveGame`) com suporte nativo a serialização binária de propriedades marcadas, sem exigir que o desenvolvedor defina o formato do arquivo. O princípio — isolar o estado que deve persistir em uma estrutura própria, separada da lógica de gameplay em tempo real — é o mesmo nas duas engines; a diferença está no grau de suporte nativo da ferramenta.

Parte 2 — SaveComponent: ponto único de gravação e leitura

Objetivo

Criar `SaveComponent` como ponto único de acesso a `BP_SaveGame` , e conectá-lo ao Event Dispatcher de coleta já existente.

Conceito

Se `BP_SaveGame` define o que é guardado, o `SaveComponent` define quem tem permissão de gravar e ler esse dado. Sem esse Component, seria tentador que cada Actor de coleta (ou o próprio Player) implementasse sua própria chamada às funções de Save/Load da engine — duplicando a mesma lógica em vários lugares do projeto e tornando qualquer mudança futura no formato de save um problema espalhado por dezenas de Blueprints.

Centralizando essa responsabilidade em um único Component, qualquer sistema do projeto que precise gravar ou ler progresso passa a depender de uma única função exposta pelo `SaveComponent` — nunca de uma chamada direta às funções nativas de SaveGame espalhada pelo projeto. Esse é exatamente o mesmo princípio de responsabilidade única já aplicado ao `InteractionComponent` (Semana 5) e ao `InventoryComponent` (Semana 10, ainda não construído).

Passo a passo

1. Na subpasta `Blueprints/Components/`, clicar com o botão direito e selecionar **Blueprint Class**.
2. Selecionar `Actor Component` como classe-pai.
3. Nomear o novo Blueprint `SaveComponent`, conforme `PROJECT_ARCHITECTURE.md`.
4. Dentro de `SaveComponent`, criar uma função `SalvarProgresso`, que recebe um `Name` (o identificador do item coletado) como parâmetro.
5. Dentro de `SalvarProgresso`, usar o nó **Create Save Game Object** (ou **Load Game from Slot**, caso já exista um save anterior) para obter uma instância de `BP_SaveGame`, adicionar o identificador recebido ao array `ItensColetados` (evitando duplicatas com um nó **Contains**) e, em seguida, usar **Save Game to Slot** para gravar essa instância em um slot nomeado (por exemplo, `"SlotPrincipal"`).
6. Criar uma segunda função, `CarregarProgresso`, sem parâmetros, que usa **Load Game from Slot** para recuperar a instância de `BP_SaveGame` do mesmo slot, e expõe o array `ItensColetados` recuperado (por exemplo, via uma variável própria do Component ou um valor de retorno).
7. Compilar e salvar `SaveComponent`.
8. Adicionar uma instância de `SaveComponent` ao `BP_Player` (Add Component, dentro do Blueprint do Player).
9. No Actor de coleta (`BP_Chest` / `BP_Pickup`), inscrever-se no próprio Event Dispatcher de coleta (ou conectar diretamente após o disparo do Dispatcher) para chamar `SalvarProgresso` no `SaveComponent` do Player, passando o `ItemRowName` do item coletado.
10. No Event `BeginPlay` do nível (ou do GameMode, conforme preferir o grupo), chamar `CarregarProgresso` no `SaveComponent` do Player, e usar o array `ItensColetados`

recuperado para reaplicar o estado (por exemplo, destruindo ou desativando os Actors de coleta cujo `ItemRowName` já conste na lista recuperada).

11. Compilar e salvar todos os Blueprints envolvidos.
12. Testar em modo Play: coletar um item, fechar o Play, e reabrir o nível, confirmando que o item coletado não aparece mais disponível para nova coleta.

Resultado esperado

`SaveComponent` funcional no `BP_Player`, gravando o identificador de cada item coletado em `BP_SaveGame` no momento da coleta, e recuperando essa lista ao carregar o nível, reaplicando corretamente o estado de progresso.

Verificando

Fechar completamente o modo Play após coletar ao menos um item, reabrir o nível e entrar em Play novamente: o Actor de coleta correspondente ao item já coletado não deve mais estar disponível para interação (ou deve indicar visualmente, ainda que de forma simples, que já foi coletado).

Problemas comuns

- **Implementar a gravação/leitura diretamente no Actor de coleta ou no Player, em vez de no `SaveComponent`** : isso duplica lógica de serialização pelo projeto. Pergunte a si mesmo: se o próximo sistema persistente (o Inventário, na Semana 10) precisar salvar dados, ele vai duplicar essa mesma lógica de gravação? Se a resposta for sim, toda leitura/escrita deve passar pelo `SaveComponent` .
- **Esquecer de verificar duplicatas antes de adicionar um item ao array `ItensColetados`** : sem essa verificação, coletar o mesmo item mais de uma vez (por erro de teste) infla o array com entradas repetidas.
- **Não testar o fluxo de fechar e reabrir o nível de fato**: apenas parar e reiniciar o Play dentro do editor, sem recarregar o nível, pode mascarar problemas de leitura que só aparecem em uma sessão nova.

Boas práticas

Nomeie o slot de save de forma explícita (`"SlotPrincipal"` , por exemplo) e mantenha esse nome centralizado — se o grupo decidir trocá-lo futuramente, a mudança deve ocorrer em um único lugar dentro do `SaveComponent` , nunca espalhada em múltiplas chamadas.

Comparação com Unity

O equivalente, em Unity, seria o desenvolvedor implementar sua própria classe de dados serializáveis (uma classe `[System.Serializable]`) e gravá-la manualmente em disco via `JsonUtility.ToJson / File.WriteAllText`, ou usar `PlayerPrefs` para casos mais simples — em ambos os casos, centralizando essa lógica em um único Manager ou serviço, para não duplicar a chamada de gravação em cada script. O princípio de centralizar o ponto único de leitura/escrita é o mesmo nas duas engines; a diferença está em que a Unreal já entrega a classe de serialização pronta (`SaveGame`), enquanto a Unity exige que o próprio desenvolvedor defina o formato do arquivo salvo.

Ao final da semana

Ao final deste encontro, o Vertical Slice deve possuir `BP_SaveGame` e `SaveComponent` funcionais, gravando e recuperando corretamente o progresso de coleta de itens de `DT_Items`, com nomenclatura conforme `PROJECT_ARCHITECTURE.md`. Isso corresponde à conclusão da linha "SaveComponent / BP_SaveGame" no roadmap de `PROJECT_ARCHITECTURE.md` (seção 6, Módulo 2). O Encontro 2 utiliza essa base para integrar todos os sistemas do módulo em um único fluxo jogável.

Desafio

Não há desafio de liberdade de solução neste encontro — a implementação do `SaveGame` é demonstração e adaptação guiada, preparando a integração final do Encontro 2.

Checklist

- ☐ `BP_SaveGame` criado a partir da classe `SaveGame`, com a propriedade `ItensColetados`
- ☐ `SaveComponent` criado a partir de `Actor Component`, com as funções `SalvarProgresso` e `CarregarProgresso`
- ☐ `SaveComponent` adicionado ao `BP_Player`
- ☐ Gravação disparada corretamente ao coletar um item (sem duplicatas no array)
- ☐ Recuperação do progresso funcionando ao carregar o nível (`BeginPlay`)
- ☐ Teste de fechar e reabrir o nível validado, com o item coletado reconhecido como já coletado

Glossário

- **SaveGame Object:** classe nativa da Unreal (`USaveGame`) dedicada à serialização e desserialização de propriedades entre sessões de jogo.
- **Slot de Save:** identificador nomeado (String) usado para gravar e recuperar uma instância específica de SaveGame em disco.
- **Save Game to Slot / Load Game from Slot:** nós de Blueprint que gravam e recuperam, respectivamente, uma instância de SaveGame em um slot nomeado.
- **SaveComponent:** Actor Component definido por PROJECT_ARCHITECTURE.md como ponto único de leitura/escrita de `BP_SaveGame` no Vertical Slice.

Referências

- EPIC GAMES. **Unreal Engine 5 Documentation** — Saving and Loading Your Game. Disponível em: <https://dev.epicgames.com/documentation/en-us/unreal-engine/saving-and-loading-your-game-in-unreal-engine>.
- EPIC GAMES. **Unreal Engine Learning Library** — introdução a SaveGame Object. Disponível em: <https://dev.epicgames.com/community/unreal-engine/learning>.
- UNITY TECHNOLOGIES. **Unity Manual** — PlayerPrefs e serialização com JsonUtility, para fins comparativos. Disponível em: <https://docs.unity3d.com/Manual/>.
- Vídeos sugeridos (apenas como apoio complementar, nunca substituindo a documentação oficial): canal oficial **Unreal Engine**, com vídeos introdutórios de Save/Load; **Mathew Wadstein**, para explicações pontuais de WTF Is? Save Game.

Imagem sugerida

Objetivo: mostrar a diferença entre estado em memória (runtime) e estado serializado (SaveGame Object), reforçando o papel do SaveComponent como ponto único de acesso.

Enquadramento: diagrama de duas colunas lado a lado, conectadas por um único ponto central. Elementos importantes: coluna esquerda — "Estado em memória (Actors, variáveis em runtime)"; coluna direita — "Estado serializado (BP_SaveGame em disco)"; centro — ícone de engrenagem rotulado "SaveComponent". O que deve ser destacado: todas as setas de gravação e leitura passam pelo bloco central do SaveComponent, nunca diretamente entre Actors e o arquivo salvo. Legenda sugerida: "Um único ponto de passagem entre o jogo em execução e o progresso salvo em disco."